Thứ Ngày Tháng Năm

P1 2.5
P2 2.5
P3 2.5
P4 2.5
10,0 2,5

Name: Ngô Trí Long Tờ 1.
Class: E25CQCE02-N
ID: N25DECE082

Q3: Doubly Linked list – Insert before Target and edge

Given a doubly linked list containing int

+ Task 1: Write pseudocode for InsertBefore (head, target, newValue) to insert a new code containing newValue immediately before the first occurrence of target

+ Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links

+ Task 3: Explicitly handle the following two edge cases in code/pseudcode.



Scanned with CamScanner

1. Target is located at the head node.
2. Target doesn't exist in the list.

1.75 Task 1 + 3: Pseudocode with edge case. (Text).

```
Function InsertBefore (head, target, newValue)
    // Edge case 2a: Empty list
    If head == NULL Then
        Return head
    End If

    // Create the new code
    newNode = CreateNode (newValue)
    // edge case 1: Target is at the head node
    If head.data == target Then
        newNode.text = newNode.Nex
        newNode.next = head
        head = newNode
        Return head
    End if.
```



Scanned with CamScanner

Thứ Ngày Tháng Năm

// Search for the target node
current = head
While current != NULL AND current.data != target Do
current = current.next
End while

// Edge case 2b: Target not found
If current == NULL Then
Free (newNode) // Avoid memory
Return head
End if

// Normal Case: Target found in the middle or end
newNode.prev = current.prev
newNode.next = current
current.prev.next = newNode
current.prev = newNode
Return head
End function



Scanned with CamScanner

Q7 Task 2: Order of pointer assignments and Justification

For a standard insertion before a current node, the exact execution order is

① newNode.prev = current.prev

② newNode.next = current

③ current.prev.next = newNode

④ current.prev = newNode

- Justification

+ Preserves existing links: Steps 1 + 2 read existing addresses without altering them.

+ Prevents lost references: Step 3 securely updates the preceding node's forward link.

+ Avoid broken chains: Step 4 updates current.prev last.

- Changing current.prev earlier breaks the path to the previous node.



Scanned with CamScanner

Scanned with CamScanner

Thứ Ngày Tháng Năm

Tiếp theo (8/2)

2.5 Q1: Singly linked list - In-Place Reversal and Pointer Tracing.

Given a singly linked list:

head → [10] → [25] → [40] → [15] → NULL

- Task 1: Write a function/algorithm in pseudocode to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function

- Task 2: Draw a diagram or step-by-step pointers trace table showing the positions and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed

Answer

1.25 Task 1:

```
Function reverseLinkedList(head)
    prev = NULL
    curr = head
    next = NULL
    While (current is not NULL) do
        next = curr.next  // Store next node
        curr.next = prev  // reverse current node's poi
        prev = curr       // Move prev forward
        curr = next       // Move curr forward
    EndWhile
    head = prev           // Update head to the new front
    Return head
End Function
```

1,25 Task 2:

In Initial State

head → [10] → [25] → [40] → [15] → NULL

prev curr next

curr

next

(prev = NULL)



Scanned with CamScanner

Thứ Ngày Tháng Năm

- After Iteration 1

NULL ⟵ [10] [25] → [40] → [15] → NULL

prev curr next

- After Iteration 2:

NULL ⟵ [10] ⟵ [25] [40] → [15] → NULL

prev curr next

- After Iteration 3:

NULL ⟵ [10] ⟵ [25] ⟵ [40] [15] → NULL

prev curr next

After Iteration 4

NULL ⟵ [10] ⟵ [25] ⟵ [40] ⟵ [15]

prev

(curr = NULL)

Final State - (head = prev)

~~head~~

head → [15] → [40] → [25] → [10] → NULL



Scanned with CamScanner

Thứ Ngày Tháng Năm

2,5 Q2: Circular Linked List – Insert at head

Given a singly ~~cu~~ circular linked list

head → [10] → [20] → [30] → (point back to head)

Task 1: Write pseudocode for the function InsertHead(head, newValue) to insert a new node containing newValue at the front of the circular linked list while maintaining its circular ~~stru~~ structure.

Task 2: Trace the execution steps (via text description or pointer diagrams) when inserting newValue = 5 into the given list

1,25 Answer

Task 2:

Initial: head → [10] → [20] → [30] → [10] (point back to head)

1. Create node: newNode = [5]
2. Find tail: Traverse to the last node, tail = [30]
3. Link front: newNode.next = head → [5] → [10]
4. Link tail: tail.next = newNode → [30] → [5]
5. Update head: head = newNode → head points to [5]

Final: head → [5] → [10] → [20] → [30] → (point back to head)



THBOOK

Scanned with CamScanner

Thứ Ngày Tháng Năm

Tiếp tục.

Q2 (Answer)

1,25 Task 1: Pseudocode

```
Function InsertHead (head, newValue).
newNode = CreateNode (newValue)
if (head == NULL) then
    newNode.next = newNode
    return newNode.
end if.
// Find tail.
tail = head
while (tail.next != head do)
    tail = tail.next
end while.
// Update pointers
newNode.next = head
tail.next = newNode
head = newNode
return head
End Function
```



Scanned with CamScanner

2,5 Q4: Sorted Linked list – In – order Insertion

Given a singly linked list sorted in ascending order

head → [2] → [5] → [8] → NULL

- Task 1: Write the function SortedInsert(head, val) to insert val into the list such that the ascending order is preserved
- Task 2: Construct a step-by-step trace table detailing pointers, iteration and condition evaluations when inserting val = 6 into the list.

1,25 Task 1: C++ Answer

```
Node* SortedInsert(Node* head, int val) {
    Node* newNode = new Node(val);
    if (!head || val < head->data) {
        newNode->next = head;
        return newNode;
    }
```



Scanned with CamScanner

Thứ Ngày Tháng Năm

```
Node * curr = head;
while (curr -> next && curr -> next -> data
< val) {
        curr = curr -> next;
    }
newNode -> next = curr -> next;
curr -> next = newNode;
return head;
}
```

1, 2 Task 2.

| Step | curr position | Condition (next -> data < 6) | Action |
|---|---|---|---|
| 1 | [2] | 5 < 6 (true) | move to [5] |
| 2 | [5] | 8 < 6 (~~true~~ false) | Stop loop |
| Insert | [5] | None | ~~List~~ Link [5] -> [6] -> [8] |



Final list [2] -> [5] -> [8] -> NULL

Scanned with CamScanner